Variables And Data Types

Variables:

Variables are containers for storing data values, or a name given to a memory location in programming.

- Instructions about Variables:
  In Python, variables do not need to be declared with any particular type.
- String variables can be declared using single or double quotes.
- Variable names are case-sensitive (a and A are different).
- Variable names must start with a letter or underscore (_).
- Variable names cannot start with a number.
- Variable names can contain letters, numbers, and underscores.
- Variable names cannot be any of the Python reserved keywords (like if, for, while etc.).

Note:

1. Python allows you to assign multiple values to multiple variables in one line.
   x, y, z = 1, 2, 3
2. You can assign a single value to multiple variables in one line.
   x = y = z = 10

Global Variable:

- Variables created outside a function are called global variables.
- Global variables can be used both outside and inside functions**.**

Local Variable:

- Variables created inside a function are called local variables.
- Local variables can only be used inside the function where they are created.

Ex ->
```
# Global variable
x = 10
def my_function():
```

```
    # Local variable
        y = 5
    print ("Inside function, x =", x)  # Accessing global variable
     print("Inside function, y =", y)  # Accessing local variable
    my_function()
    print("Outside function, x =", x)  # Global variable accessible here
#print("Outside function, y =", y)  # This will cause an error because y is local
```

Python Data Types:

Datatypes with Categories:

Text Type

- str → String

Code : name = "Python" print(name, "=>", type(name))

Numeric Type

- int, float, complex

code : Num = 100 , print(Num, "=>", type(Num)) , pi = 3.14 , print(pi, "=>", type(pi)) , z = 2 + 3j , print (z, "=>", type(z))

Sequence Type

- list, tuple, range

code : fruits = ["apple", "banana", "cherry"] print(fruits, "=>", type(fruits)) , coordinates = (10, 20) , print(coordinates, "=>", type(coordinates),)
r = range(5) , print(r, "=>", type(r))

Mapping Type

- dict

code : nums = {1, 2, 3} print(nums, "=>", type(nums))

Set Type

- set, frozenset

code : nums = {1, 2, 3} print(nums, "=>", type(nums))
fnums = frozenset([1, 2, 3]) , print(fnums, "=>", type(fnums))

Boolean Type

- bool

code : is valid = True , print(is_valid, "=>", type(is_valid))

Binary Type

- bytes, byte array, memory view

Code : b = b'hello' print(b, "=>", type(b))
ba = bytearray([65, 66, 67]) , print(ba, "=>", type(ba))
mv = memoryview(b"hello") , print(mv, "=>", type(mv))

None Type

- None

Code : nothing = None , print(nothing, "=>", type(nothing))

Data Type:

Int
float
string
Boolean
None

Note:

When we want to know **which type** of variable is declared in our code, we can use the `type ()` function.
The `type ()` function is used to return the **data type** of a variable or object.

Code:
```
x = 10
print(type(x))  # Output: <class 'int'>
y = "Hello"
print(type(y))  # Output: <class 'str'>
```

Practice Set

1. Declares **three variables**: a string, an integer, and a float.
2. Prints their values and their data types using the type() function.
3. Assigns the same value to **three variables** in a single line.
4. Creates a function that uses a global variable and declares a local variable inside the function.
5. Tries to access the local variable outside the function and observes the result.